LAB 04 Decisions

October 26, 2026

```
[ ]: # We first import from a useful packages that will be required in this lab
     import numpy as np
     from random import seed
     from random import random
```

1 Decisions - if, elif, else

Decisions are made using `if` statements, and we will get practice writing code to make decisions. But first we will get some practice at writing **functions** to make the decision code clearer by shuffling off complexity elsewhere in our code. We will also need something to make decisions about - and we will choose to do that in this lab by rolling computer dice. So we shall also do a little work on *pseudo-random number generation* to create some numbers we can make decisions upon.

1.0.1 Functions

Functions contain lines of python code that can be called from anywhere else in the notebook. Functions accept *input parameters* and `return` a *result*. You have already explored many built-in examples of functions, e.g. the `np.sin()` function, which takes one input argument (an angle in radians) and returns one output argument (the sine of that angle).

Each notebook can only see certain functions:

1. functions core to Python, e.g. `print()`
2. functions imported into the notebook, i.e. `import numpy as np` means the notebook can see all NumPy functions
3. functions defined by you within the notebook itself.

This means that if you define your own custom function in one notebook, a separate notebook will not know about that function (unless you also define it in the new notebook).

The `total` function defined below, for example, is a simple Python function that calculates the sum of the elements of a 1D array:

```
def total(x):  # def allows us to DEFine a new function called total that takes one input 'x'
    # Everything that is indented here is a part of the function
    ''' TOTAL - sum of all elements in 1D array x
        This function works for row and column vectors of any length.
        Here we have chosen the output argument to be s,
        but you can change that to whatever you like.'''
```

```
    s=0  # initialise our sum variable
    for i in np.arange(x.size):
        s = s + x[i]
    return s  # return allows the user to get the value of s

# The next lines are not inside the function (note that they are not indented),
# so they cannot see anything from within the function
myArray = np.random.rand(10,1)  # create a 10 item 1D array of random values
myTotal = total(myArray)  # determine the total value using our custom function
print(myTotal)  # print the result
```

Note the use of a multi-line comment (surrounded in triple quotes) that gives a description of what the function does. Without such useful comments my colleague may not understand the function, or worse - I might forget when I come back to the codes in a year's time.

Try predicting the results from the following function examples, before running the Python code.

[ ]:
```
def catYears(human_years):
    '''Calculate cat years from human years.'''
    y = human_years / 7.0
    rounded = int(y+0.5)
    return rounded

def sumOfSquares(a,b):
    '''Must have a use somewhere.'''
    return a*a + b*b

def p(years, c, animal):
    '''Just output, no more than that'''
    print("If I were a {}, I'd be ".format(animal))
    print("{} years old.".format(years))
    print("The sum of squares ")
    print("was {} units.".format(c));

my_age = 18    # set up some variables
x = 3.3
animal = "cat"

years_a = catYears(my_age)    # and then use them in functions
sum = sumOfSquares(x,7.4)
p(years_a, sum, animal)
```

1.0.2 Random number generation

The `random` library includes useful functions dealing with pseudorandom numbers. When people write **random number** with regards to computers, they really mean **pseudorandom number**. The basic problem is that if you start from a single value (known as the *random seed*), and do a series of mathematical operations on it, you will not get an actual random number – every time

you start from the same random seed, you will get the same number!

To quote one of the giants in computer science,

> Anyone who attempts to generate random numbers by deterministic means is, of course, living in a state of sin. - John von Neumann

However, pseudorandom numbers are actually really useful. They have many of the properties of real random numbers, but if you choose the same sequence each time, debugging your code can be a lot easier.

Try the code below, changing the seed, to see how this works.

[ ]:
```
from datetime import datetime
from random import seed
from random import random

seed(1)          # set up the random number generator
print(random())  # print random number between 0 (inclusive) and 1.0␣
 ↪(exclusive)
# try the number datetime.now() if you want an unrepeatable sequence
```

2 Exercise : Writing your own Functions

Create a *die rolling* game. You are probably familiar with 6-sided dice, but some games use dice with 4, 6, 8, 10, 20, and 100 sides. You will make the computer pick a random number for a 6-sided die and a 20-sided die.

Write a `rollDie(...)` function to get a random value. The function should have one integer argument for the value of the die `(number_of_sides)`. Use the `random()` function (*yes, you will be writing a function that calls another function*) to get a float between 0 and 0.999...) and do some maths to transform that into an integer between 1 and `number_of_sides` inclusive)

Your main program should be be the following `python value = rollDie(6) print("6-sided die: {}".format(value)) value = rollDie(20) print("20-sided die: {}".format(value))`

[ ]:
```
# Die Rolling Game

# libraryimports set up in the cells above
# student defined function in here



value = rollDie(6)
print("6-sided die: {}".format(value))
value = rollDie(20)
print("20-sided die: {}".format(value))
```

2.1 Decision Blocks

The logic of an `if <expression>:` statement is fairly simple. If the expression is true, an indented code statement is run. Closely relared commands are `if <expression>: ... else:`, and `if <expression>: ... elif <expression: ... else:` which act precisely as you would expect.

Note that `=` and `==` mena different things in Python. The former may be read as *becomes equal to* and indicates variable assignment. The latter is *is equal to* and is a logical expression returning `True` if the two numbers are equal.

The logical operators `not`, `and` and `or` are particularly useful with `if`. Parentheses () are highly encouraged when using logical operators.

3 Exercise - a simple guessing game (5 marks)

Create a **guessing game** program. Your program should:

- Make the computer pick a random number to be the answer. It should be an integer between 1 and 32 (inclusive). You may use the function that you worte previously.
- Write a function for the user's guess. This function must:
  - have one integer argument (correct_answer),
  - read an integer from the keyboard,
  - check the entered number against the correct answer,
  - output the appropriate message ("correct" / "too high" / "too low"),
  - return a `True` if the user's guess was correct, and `False` if the user's guess was wrong.
- Give the user 5 chances to guess the right answer. (hint: you will write a function that checks a guess. What happens if you copy and paste that function call 5 times?)
- End the game if the user is correct, or if s/he's used up all 5 chances. Print either a "you win" or "you lose", as appropriate.
- You may not use any loops for this program !

```
[ ]: # Write your code here
```